

# Прийоми ефективного програмування

## Частина 2. Декоратори

### Поняття декоратора

У Python функції можуть бути аргументами і результатами виконання інших функцій. Ця можливість покладена в основу функціонування декораторів. Декоратор дає змогу змінювати поведінку функції, змінюючи або не змінюючи саму функцію. Іншими словами, декоратор підміняє одну функцією іншою з тією ж самою назвою.

Інтерпретатор Python розпізнає декоратор за символом @. Синтаксис декорування функції виглядає так:

```
@some_decorator
def some_function():
    pass # тіло функції
```

Цей запис аналогічний наступному:

```
def some_function():
    pass # тіло функції

def some_decorator(func):
    pass # тіло декоратора

some_function = some_decorator(some_function)
```

Тобто, декоратор — це функція, аргументом якої є інша функція. Результатом виконання декоратора теж має бути функція.

Перший варіант запису, попри свою лаконічність, не завжди кращий. Оскільки функція декорується безпосередньо під час оголошення, то доступ до оригіналу (недекорованої функції) ускладнюється. Тому інколи доцільніше використовувати другий варіант декорування.

З попередніх фрагментів коду не зрозуміло, що ж міститься всередині функції-декоратора `some_decorator()`. Зазвичай там оголошується нова функція-обгортка, яка не модифікує невідворотно оригінальну функцію, а тільки змінює її поведінку у випадку декорування. Продемонструємо це на конкретному прикладі.

## Декорування функцій

Нехай маємо функцію `money()`, яка повертає ціле число, яке дорівнює сумі коштів на рахунку через `years` років при початковому внеску `deposit` та `pc` складних річних відсотках. Припустимо, з певних міркувань нам потрібно, щоб ця функція повертала не просто суму коштів як число, а з вказуванням грошової одиниці. Щоби не змінювати тіла функції (можливо, вона ще колись буде потрібна і в початковому варіанті), її декорують. Напишемо декоратор, який дописує до результату функції "грн." та застосуємо його до нашої функції:

```
# декоратор
def uah(func):
    def wrapper(*args, **kwargs):
        return str(func(*args, **kwargs)) + ' грн.'
    return wrapper

# декорування
@uah
def money(deposit, years, pc):
    return round(deposit * (1 + pc/100)**years)

print(money(1000, 2, 10))      # буде надруковано 1210 грн.
```

Функція-декоратор `uah()` змінює її поведінку так, що вона повертатиме цю суму у вигляді рядка, який утворений конкатенацією суми коштів на рахунку та скороченим позначенням української гривні. В обгортці `wrapper` використано параметри `*args` і `**kwargs`, оскільки декоратору нічого невідомо про аргументи вхідної функції, а він зазвичай повинен декорувати різні функції з різною кількістю аргументів. Тобто, наш декоратор можна застосовувати не лише до функції `money()`, а, взагалі кажучи, до результату виконання будь-якої функції — цей результат буде перетворений в рядок і конкатенований з рядком "грн."

Як зазначалося раніше, після декорування функції втрачається прямий доступ до її атрибутів:

```
print(money.__name__)          # wrapper
```

Щоб цього не ставалося, рекомендується всередині свого декоратора використовувати декоратор `functools.wraps` стандартної бібліотеки Python, який переносить метадані з недекованого об'єкту (у нашому випадку — функції) в декорований. Так, декоратор `uah` набуде вигляду

```
import functools
```

```
def uah(func):  
    @functools.wraps(func)  
    def wrapper(*args, **kwargs):  
        return str(func(*args, **kwargs)) + ' грн.'  
    return wrapper
```

Тепер після декорування функції доступ до її атрибутів (але не до виклику!) не буде втрачено:

```
print(money.__name__)          # money
```

Якщо ж потрібен одночасно доступ до обох варіантів функції — декорованого та недекованого, то потрібно використовувати розгорнуту версію декорування (без використання @):

```
def uah(func):  
    def wrapper(*args, **kwargs):  
        return f"{func(*args, **kwargs)} грн."  
    return wrapper  
  
def money(deposit, years, percent):  
    return round(deposit * (1 + percent / 100) ** years)  
  
initial_money = money # запам'ятовуємо недековану функцію  
money = uah(money) # декоруємо функцію  
  
print(money(1000, 2, 10))          # 1210 грн.  
print(initial_money(1000, 2, 10))  # 1210
```

Напишемо ще один декоратор, який виводитиме час виконання функції, яку він декорує.

```
def timer(func):  
    from time import time  
    def wrapper(*args, **kwargs):  
        start = time()  
        result = func(*args, **kwargs)  
        print(f"Execution time: {time()-start} seconds")  
        return result  
    return wrapper
```

Він може бути застосований до будь-якої функції, наприклад:

```
@timer
def length_of_factorial(x):
    return len(str(factorial(x)))

print(length_of_factorial(100000))

# результат виконання:
Execution time: 16.984936237335205 seconds
456574
```

До функцій можна застосовувати одночасно декілька декораторів, які викликатимуться у зворотному порядку:

```
@timer
@uah
def money(deposit, years, percent):
    return round(deposit * (1 + percent / 100) ** years)

print(money(1000, 2, 10))

# результат виконання:
Execution time: 2.7894973754882812e-05 seconds
1210 грн.
```

## Декоратори з параметрами

Python дає можливість передавати декораторам параметри. У такому разі матимемо три рівні вкладення функцій.

Реалізуємо декоратор, який дописує в кінець результату виконання функції не конкретну грошову одиницю (як було в нашому випадку з "грн."), а задану параметром (аргументом декоратора):

```
def currency(currency):
    def wrapper(func):
        def inner(*args, **kwargs):
            return f"{func(*args, **kwargs)} {currency}"
        return inner
    return wrapper

@currency('USD')
def money(deposit, years, percent):
    return round(deposit * (1 + percent / 100) ** years)
```

```
print(money(1000, 2, 10))      # 1210 USD
```

## Момент виконання декораторів

Інтерпретатор виконує декоратор одразу після оголошення функції, яку він декорує (виконайте та проаналізуйте результат виконання цього коду):

```
registry = []

def register(func):
    print(f'running register({func.__name__})')
    registry.append(func)
    return func

@register
def f1():
    print('running f1()')

@register
def f2():
    print('running f2()')

def f3():
    print('running f3()')

def main():
    print('running main()')
    print('registry ->', registry)
    f1()
    f2()
    f3()

main()
```

## Декорування методів класу

Оскільки методи є за своєю суттю нічим іншим, ніж функціями, просто описаними всередині класу, то декорування методів відбувається аналогічно до декорування функцій.

Python надає низку вбудованих декораторів, які часто використовують для декорування методів. Два з них ми вже використовували під час вивчення об'єктно-орієнтованого програмування, а саме під час створення геттерів і сеттерів. Розглянемо ще три декоратори: `@classmethod`, `@staticmethod` і `@property`.

### Декоратор `@classmethod`

Нехай ми маємо такий клас:

```
class Class:

    def doubler(cls, x):
        print(x**2)
```

Припустимо, ми хочемо, щоб метод `doubler()` був доступний як атрибут класу, а не екземпляра (саме тому ми замість `self` написали `cls`, хоча формально перший аргумент може мати будь-яке ім'я, а інтерпретатор розпізнаватиме його як `self`). Тому в такій реалізації це неможливо. Виклик методу через клас

```
Class.doubler(5)
```

спричинить помилку

```
TypeError: doubler() missing 1 required positional argument: 'x'
```

Для коректної роботи таких методів (вони називаються методами класу) використовується декоратор `@classmethod`:

```
class Class:

    @classmethod
    def doubler(cls, x):
        print(x**2)
```

```
Class.doubler(5)      # буде надруковано 25
```

Метод класу буде також доступний і через екземпляр:

```
c = Class()
c.doubler(5)      # буде надруковано 25
```

## Декоратор @staticmethod

Нехай тепер ми маємо клас з статичним методом (методом, який не залежить від екземпляра):

```
class Class:

    def doubler(x):
        print(x**2)
```

Тепер ми не матимемо до нього доступ через екземпляр (тільки через клас). Виклик

```
c = Class()
c.doubler(5)
```

спричинить помилку

```
TypeError: doubler() takes 1 positional argument but 2 were given
```

Для коректної роботи таких методів використовується декоратор @staticmethod:

```
class Class:

    @staticmethod
    def doubler(x):
        print(x**2)
```

Статичний метод буде також доступний і через клас.

## Декоратор @property

Декоратор `@property` використовується для перетворення методу в захищений від змін атрибут (властивість). Продемонструємо його на прикладі:

```
class Person:

    def __init__(self, first_name, last_name):
        self.first_name, self.last_name = first_name, last_name

    @property
    def full_name(self):
        return f'{self.first_name} {self.last_name}'

teacher = Person('Roman', 'Seliverstov')
```

Тепер для отримання повного імені не потрібно викликати метод (дописувати `()` в кінці), а звертатися до нього як до атрибута-властивості:

```
print(teacher.full_name)          # буде надруковано Roman Seliverstov
```

А при спробі зміни

```
teacher.full_name = 'R. Seliverstov'
```

виникне помилка:

```
AttributeError: can't set attribute
```

## Декорування класів

Декорувати можна не тільки функції, а й класи. У цьому випадку обгорткою виступатиме клас, наприклад:

```
def decorator(cls):

    class Wrapper:

        def __init__(self, *args, **kwargs):
            self.wrapped = cls(*args, **kwargs)

        def __getattr__(self, name):
            return getattr(self.wrapped, name) * 3
```



```

        return Wrapper

#@decorator
class A:
    def __init__(self, x, y):
        self.z = 'spam'

a = A(1, 2) # викликається Wrapper(1, 2)
print(a.z) # викликається Wrapper.__getatr__(z)

```

У результаті буде надруковано `spamspamspam`. Якщо клас не декорувати, то буде надруковано `spam`. Наш декоратор не зачіпає функціональності конструктора декорованого класу, а тільки змінює геттер атрибутів (фактично потроєє їхні значення).

## Класи-декоратори

Декоратором може бути не функція, а клас. У цьому випадку роль обгортки виконує метод `__call__()`.

Нижче показано, як написати використовуваний нами декоратор `@timer` у вигляді класу, а не функції:

```

from time import time

class Timer:
    def __init__(self, func):
        self.func = func

    def __call__(self, *args, **kwargs):
        start_execution_time = time()
        result = self.func(*args, **kwargs)
        self.execution_time = time() - start_execution_time
        print(f'Time of execution: {self.execution_time} seconds')
        return result

@Timer
def squares_comprehension(n):
    return [x**2 for x in range(n)]

squares_comprehension(10**7)

```

Результатом буде час генерування списку квадратів перших 10000000 натуральних чисел, починаючи з 0.

Time of execution: 4.100403070449829 seconds